CSS դիրքավորում և էջի դասավորություն

Արձագանքող վիզայն տեսողական էֆեկտների և *Sass SCSS*

CSS-ը սահմանում է վեբ էջի տարրերի արտաքին տեսքը և տարածական դասավորությունը: Էջի կառուցվածքը ձևավորվում է HTML տարրերով, իսկ CSS-ը որոշում է դրանց չափերը, դիրքը, փոխադարձ հեռավորությունները և վարքը տարբեր էկրանների դեպքում: Նույն HTML կառուցվածքը կարող է ներկայացվել մեկ սյունակով, քարտերի ցանցով կամ նավիգացիայի հորիզոնական շարքով՝ կախված կիրառված դասավորության կանոններից:

Տարրերի տեղադրումը հասկանալու հիմքը սովորական հոսքն է: Դիրքավորման հատկությունները փոխում են տարրի կապը այդ հոսքի հետ, Flexbox-ը և Grid-ը կազմակերպում են տարրերի խմբերը, իսկ Media Queries-ը թույլ են տալիս դասավորությունը կապել դիտման միջավայրի պայմաններին: Անցումները, փոխակերպումներն ու անիմացիաները լրացնում են էջի տեսողական վարքը: Sass/SCSS-ը ոճերի սկզբնական կոդը կազմակերպելու լրացուցիչ միջոց է, որի արդյունքը սովորական CSS է:

Տարրերի դիրքավորում և հոսք

Սովորական հոսք և relative

Էջի տարրերն սկզբում դասավորվում են սովորական հոսքով: Բլոկային տարրը սովորաբար սկսում է նոր տողից, իսկ տեքստն ու inline տարրերը տեղավորվում են տողերի մեջ: position: static-ը լռելյայն դիրքավորումն է: Այդ դեպքում top և left հատկություններով տարրը չենք տեղաշարժում: relative-ի դեպքում տարրը նախ ստանում է իր սովորական տեղը, հետո տեսանելիորեն տեղաշարժվում է տրված չափով:

Օրինակում երկրորդ բլոկը տեղափոխվում է 18px ներքև և 24px աջ: Նրա սկզբնական զբաղեցրած տեղը պահպանվում է, հետևաբար երրորդ բլոկը չի բարձրանում այդ տեղը լրացնելու համար: Տեղաշարժը կարող է ծածկել հարևան տարրին: relative-ը հաճախ պետք է նաև որպես ներսի absolute տարրի հենակետ:

```
.box {
  position: relative;
  top: 18px;
  left: 24px;
}
```

Absolute և հենակետ

absolute տարրը դուրս է գալիս սովորական հոսքից: Մյուս տարրերը նրա համար առանձին տեղ չեն պահում: Սկզբնական սովորեցման համար հարմար կանոնն է՝ քարտին տալիս ենք relative, նրա նշանին՝ absolute: Այդպես նշանի top և right հեռավորությունները հաշվում ենք քարտի դիրքավորման տարածքից: Եթե մոտակա նախնին դիրքավորում չունի, պետք է փնտրել ավելի վերևի համապատասխան հենակետը:

Պետք է հաշվի առնել, որ transform-ի նման որոշ հատկություններ նույնպես կարող են հենակետ ձևավորել: Ուստի absolute-ը միշտ անմիջական ծնողի նկատմամբ է արտահայտությունը սխալ է: Նշանը չի բարձրացնում քարտի բարձրությունը: Եթե այն ծածկում է վերնագիրը, քարտին պետք է ավելացնել համապատասխան padding կամ ընտրել այլ տեղադրում:

```
.card { position: relative; }
.badge {
  position: absolute;
  top: 12px;
  right: 12px;
}
```



Fixed դիրքավորում

Fixed դիրքավորումը հարմար է օգնության կոճակի կամ էկրանին մշտապես երևացող կառավարման տարրի համար: Տարրը դուրս է հոսքից և սովորաբար տեղավորվում է viewport-ի նկատմամբ: Էջը ոլորելիս այն նույն տեղում է մնում: Քանի որ հոսքը նրա համար տեղ չի պահում, ներքևի բովանդակությունը կարող է հայտնվել կոճակի տակ:

Օրինակում 20px հեռավորությունը հաշվվում է պատուհանի աջ և ներքևի եզրերից: Այս կանոնն ունի բացառություն. transform ունեցող նախնին կարող է փոխել fixed-ի հենակետը: Մշտական վերնագրի դեպքում բովանդակության վերևում պետք է նախատեսել բավարար տարածք: Փոքր էկրանին մեծ fixed բլոկը կարող է զբաղեցնել ընթերցման տարածքի զգալի մասը:

```
.help {
  position: fixed;
  right: 20px;
  bottom: 20px;
  z-index: 20;
}
```

Sticky և ոլորում

Sticky տարրը տեղ է զբաղեցնում սովորական հոսքում: Երբ ոլորման ընթացքում հասնում է top-ով նշված շեմին, մնում է այդ սահմանին մոտ՝ իր պարունակող բլոկի թույլատրած շրջանակում: Այն չի կարող անվերջ դուրս գալ իր ծնողի սահմաններից: Օրինակի top: 0-ը նշանակում է կպչել վերին եզրին:

Եթե արդյունքը չի երևում, ստուգում ենք երեք բան. կա՞ top կամ այլ համապատասխան inset, կա՞ բավական երկար բովանդակություն, և ո՞ր նախնին է ստեղծում ոլորման միջավայրը: Նախնու overflow: auto կամ hidden արժեքները կարող են փոխել sticky-ի վարքը: Եթե նավիգացիայի ծնողն ունի միայն նավիգացիայի բարձրությունը, կպչելու համար ազատ տարածք կարող է չլինել: Ֆոնն անհրաժեշտ է, որպեսզի ներքևից անցնող տեքստը չխառնվի նավիգացիայի հետ:

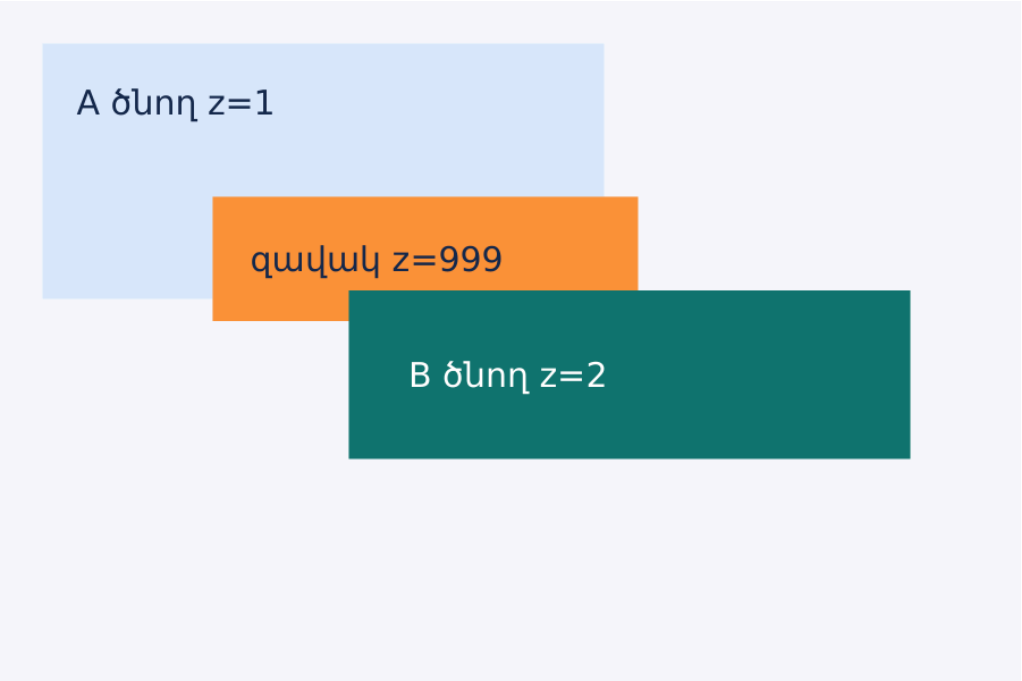
```
.site-nav {
  position: sticky;
  top: 0;
  z-index: 10;
  background: white;
}
```

z-index և շերտավորման միջավայր

z-index-ը մասնակցում է տարրերի շերտավորման հերթականությանը: Դիրքավորված տարրերի համար ոչ auto z-index-ը կարող է ստեղծել առանձին stacking context: Այդ միջավայրի ներսում զավակների թվերը համեմատվում են իրար հետ, իսկ ամբողջ միջավայրը արտաքին հարևանների համար հանդես է գալիս որպես մեկ ամբողջություն:

Օրինակում group-a-ի զավակն ունի 999, բայց իր ծնողի միջավայրը group-b-ից ցածր է: Այդ զավակը միայն մեծ թիվ ստանալով չի անցնի group-b-ի վրայով: transform-ը և 1-ից փոքր opacity-ն նույնպես կարող են միջավայր ստեղծել: Flex և Grid տարրերի վրա z-index-ը կարող է գործել նաև առանց position-ի փոփոխության: Խնդիրը շտկելիս ստուգում ենք նախնիներին, ոչ միայն ծածկվող տարրի թիվը:

```
.group-a {
  position: relative; z-index: 1;
}
.group-a .badge {
  position: absolute; z-index: 999;
}
.group-b {
  position: relative; z-index: 2;
}
```



Float clear և flow-root

Float-ը թույլ է տալիս պատկերը կամ այլ բլոկը տեղավորել ձախ կամ աջ, իսկ հաջորդ տեքստը հոսում է դրա կողքով: Այն օգտակար է հոդվածում պատկերն ու պարբերությունները համաղրելու համար: Float տարրը դուրս է սովորական բլոկային հոսքից, բայց ազդում է հարևան տեքստի տեղավորման վրա: Սա տարբերվում է absolute-ից, որը նույն կերպ տեքստ չի շրջանցում:

clear: both-ը համապատասխան բլոկը տեղաղրում է իրեն նախորդող ձախ և աջ float-երից ներքև: Այն չի չեղարկում float-ը: Եթե ծնողը միայն float զավակներ ունի, նրա բարձրությունը կարող է չներառել դրանք: display: flow-root-ը ստեղծում է բլոկային ձևավորման նոր միջավայր և ծնողին թույլ է տալիս ներառել իր float-երը: Ամբողջ էջի հիմնական դասավորությունը նոր նախագծում հարմար է անել Flexbox-ով կամ Grid-ով:

```
.photo {
  float: left; width: 130px;
  margin: 0 18px 12px 0;
}
.next { clear: both; }
.article { display: flow-root; }
```

Flexbox դասավորություն

Flexbox ծնող և առանցքներ

Flexbox-ը սկսում ենք ծնող տարրից: display: flex ստացած տարրի անմիջական զավակները դառնում են flex items: Նրանց ներսի բոլոր ժառանգները ինքնաբերաբար flex items չեն դառնում: Հիմնական առանցքի ուղղությունը որոշում է flex-direction-ը, իսկ լայնական առանցքը նրան ուղղահայաց է:

Սովորական հորիզոնական գրության և ձախից աջ ուղղության դեպքում row-ը դասավորում է ձախից աջ, իսկ column-ը՝ վերևից ներքև: Հետագայում justify-content-ը վերաբերում է հիմնական առանցքին, align-items-ը՝ լայնականին: Ուստի դրանք միշտ հորիզոնական և ուղղահայաց կոչելը սխալ պատկերացում է ստեղծում: flex-direction-ի փոփոխության դեպքում փոխվում է նաև հիմնական և լայնական առանցքների ուղղությունը:

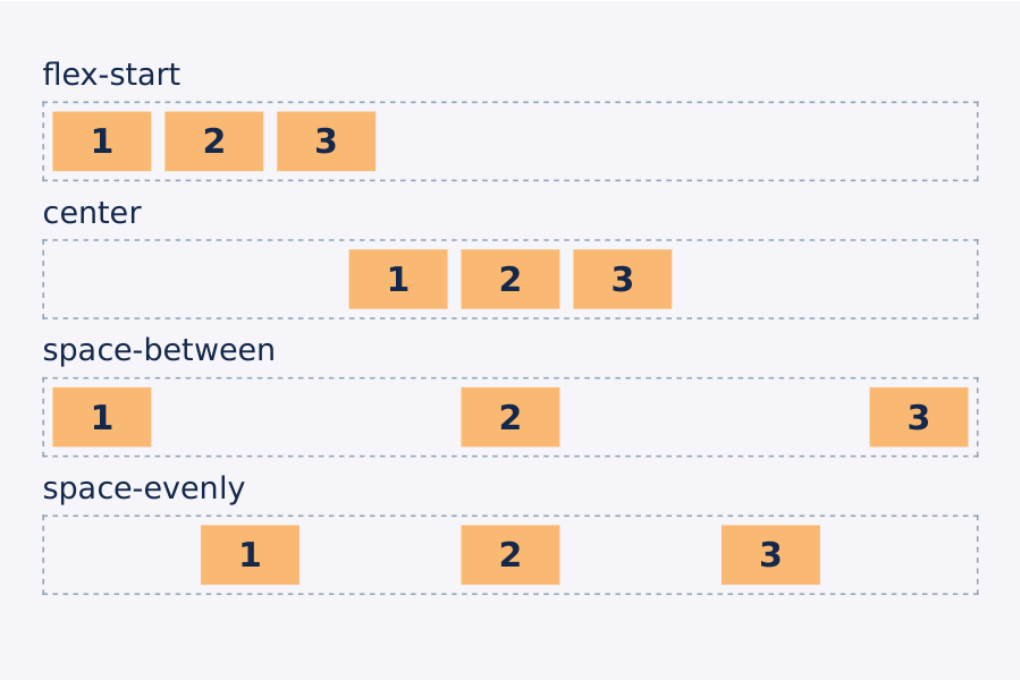
```
.row {
  display: flex;
  flex-direction: row;
  gap: 16px;
}
.column { flex-direction: column; }
```

justify-content և բաշխում

justify-content-ը բաշխում է հիմնական առանցքի ազատ տարածքը: flex-start-ը տարրերը հավաքում է սկզբում, center-ը՝ կենտրոնում, flex-end-ը՝ վերջում: space-between-ը առաջին և վերջին տարրերը մոտեցնում է եզրերին, իսկ ազատ տարածքը բաժանում է միջակայքերին: space-around-ի դեպքում եզրային բացատը մոտ կեսն է ներքինի, space-evenly-ի դեպքում միջակայքներն ու եզրային բացատները հավասար են:

Եթե տարրերը արդեն զբաղեցնում են ամբողջ հասանելի լայնությունը կամ flex-grow-ով կլանում են այն, բաշխման տարբերությունը կարող է չերևալ: gap-ը առանձին նվազագույն միջտարրային բացատ է, իսկ space-between-ը կարող է դրան ավելացնել ազատ տարածք: Բավական լայն ծնողի մեջ նույն երեք բլոկների բաշխումը փոխվում է ընտրված արժեքին համապատասխան:

```
.nav {
  display: flex;
  justify-content: space-between;
}
/* այլ արժեքներ */
/* flex-start, center, flex-end */
/* space-around, space-evenly */
```



align-items և align-self

align-items-ը սահմանում է նույն flex տողի տարրերի լայնական հավասարեցումը: center-ը կենտրոնացնում է, flex-start և flex-end արժեքները մոտեցնում են համապատասխան եզրին: stretch-ը ձգում է ավտոմատ լայնական չափ ունեցող տարրերը: Եթե row ուղղության տարրին հստակ height է տրված, այն նույն կերպ չի ձգվի: baseline-ը հարմար է տարբեր տառաչափերով տեքստերի տողահիմքերը հավասարեցնելու համար:

align-self-ը փոխում է մեկ կոնկրետ տարրի հավասարեցումը: Օրինակում առաջին երկու տարրերը կենտրոնում են, իսկ special-ը իջնում է ներքև: Փորձն ավելի տեսանելի կլինի, եթե ծնողն ունենա տարրերից մեծ բարձրություն: align-content-ը այլ խնդիր է լուծում՝ բազմատող flex միջավայրի տողերի բաշխումը, երբ լայնական ազատ տարածք կա:

```
.row {
  display: flex;
  align-items: center;
  height: 160px;
  gap: 16px;
}
.special { align-self: flex-end; }
```

Փաթաթում և gap

Լռելյայն flex-wrap-ը nowrap է: Տարրերը մնում են մեկ տողում, իսկ flex-shrink-ի պատճառով կարող են նեղանալ: wrap-ը թույլ է տալիս տարրերի մի մասն անցկացնել հաջորդ տող: Ամեն տող ինքնուրույն flex տող է: Սա կարևոր տարբերություն է Grid-ից. երկրորդ flex տողի տարրերը պարտադիր չեն հավասարվել առաջին տողի սյուների տակ:

gap: 12px 20px-ը row ուղղությամբ օրինակին տալիս է 12px բացատ տողերի և 20px՝ սյուների միջև: Այն ծնողի արտաքին եզրին padding չի ստեղծում: Օրինակում flex: 0 0 140px-ը կանխում է աճն ու նեղացումը և տալիս սկզբնական 140px չափ: Այս սահմանմամբ հաջորդ տարրը տեղավորվում է նույն տողում միայն այն դեպքում, երբ դրա և միջակայքի համար բավարար ազատ տարածք կա:

```
.tags {
  display: flex;
  flex-wrap: wrap;
  gap: 12px 20px;
}
.tag { flex: 0 0 140px; }
```

Flex տարրի աճ և նեղացում

flex-grow-ը որոշում է դրական ազատ տարածքից ստացվող բաժինը: flex-shrink-ը մասնակցում է պակասող տարածքի հաշվին նեղացմանը, իսկ flex-basis-ը տալիս է չափի հաշվարկի սկզբնական հիմքը: flex: 1 1 0-ը նշանակում է աճել, անհրաժեշտության դեպքում նեղանալ և հաշվարկը սկսել զրոյական հիմքից:

Եթե երկու ճկուն տարրերի grow արժեքները 1 և 2 են և հիմքերը զրո, ազատ տարածքը նրանց միջև բաժանվում է այդ հարաբերությամբ՝ հաշվի առնելով այլ չափային սահմանափակումները: min-width: 0-ը հաճախ պետք է երկար տեքստ ունեցող flex տարրի նեղացումը թույլ տալու համար: order-ը փոխում է տեսանելի հերթականությունը, սակայն չի վերագրում HTML-ի իմաստային կամ ստեղնաշարային կարգը: Ուսուցման օրինակներում պահում ենք տրամաբանական HTML հերթականությունը:

```
.menu { flex: 0 0 140px; }
.content {
  flex: 1 1 0;
  min-width: 0;
}
.note { flex: 2 1 0; }
```

Նավիգացիա Flexbox-ով

Այս օրինակում նավիգացիան ունի երկու անմիջական զավակ՝ կայքի անունը և հղումների ul ցանկը: Առաջին Flexbox-ը կառավարում է այդ երկու խմբերի հարաբերությունը: Երկրորդ Flexbox-ը աշխատում է ul-ի ներսում և դասավորում է li տարրերը: Այդ երկու մակարդակները տարբեր պատասխանատվություն ունեն:

space-between-ը անունն ու ցանկը հեռացնում է իրարից, align-items: center-ը հավասարեցնում է լայնական ուղղությամբ, իսկ wrap-ը թույլ է տալիս նեղ էկրանին խմբերը տեղափոխել: UI-ի margin և padding լռելյայն արժեքները հեռացնում ենք, որպեսզի բացատները վերահսկվեն մեր կողով: Հղումները պետք է մնան իրական a տարրեր՝ աշխատող href-ով և տեսանելի focus վիճակով: Առաջին տարբերակում բարդ բացվող մենյուի կարիք չկա:

```
.nav {
  display: flex; flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
}
.links {
  display: flex; flex-wrap: wrap;
  gap: 16px; list-style: none;
  margin: 0; padding: 0;
}
```

CSS Grid և Flexbox համատեղում

Grid տողեր սյուներ և գծեր

Grid-ը կազմակերպում է միաժամանակ տողերն ու սյուները: Ծնողին տալիս ենք display: grid, ապա սահմանում սյուների և անհրաժեշտության դեպքում տողերի չափերը: Նրանց միջև ընկած գծերով կարող ենք նշել տարրերի սկիզբն ու ավարտը: Երեք սյուների համար կան չորս սահմանային գծեր՝ 1, 2, 3 և 4:

fr-ը ցանցի հասանելի մնացորդային տարածքի բաժինն է: gap-ը նախ զբաղեցնում է իր տարածքը, հետո ճկուն սյուները բաժանում են մնացածը: Եթե տողեր չենք սահմանում, բովանդակության ավելանալիս առաջանում են ավտոմատ տողեր: Վեց բլոկից բաղկացած օրինակում յոթերորդ տարրի ավելացումը ստեղծում է նոր տող: Նա կանցնի նոր տող, իսկ այդ նոր տողի չափը այլևս մեր նշած առաջին երկու տողերից չէ:

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 80px 100px;
  gap: 16px;
}
```

repeat և minmax

repeat(3, ...) գրառումը երեք անգամ կրկնում է նեղսի սահմանումը: minmax(0, 1fr)-ը տալիս է զրոյական նվազագույն և մեկ ճկուն բաժնով առավելագույն չափ: Այն օգտակար է, երբ ուզում ենք, որ երկար բովանդակության նվազագույն լայնությունը չպարտադրի լայն սյունակ: Բովանդակության սեփական overflow-ի հարցը միևնույն է պետք է լուծել:

minmax(220px, 1fr)-ը արդեն պահանջում է առնվազն 220px սյունակ: Եթե երեք այդպիսի սյունակին ավելացնենք երկու gap, անհրաժեշտ ընդհանուր լայնությունը մեծանում է: Նեղ ծնողի մեջ հաստատուն երեք սյուները կարող են արտահոսք տալ: Հաջորդ քայլը auto-fit-ն է, որը կարող է փոփոխել սյուների քանակը: Թվերի օրինակով հաշվեք՝ 3 × 220 + 2 × 16 = 692px:

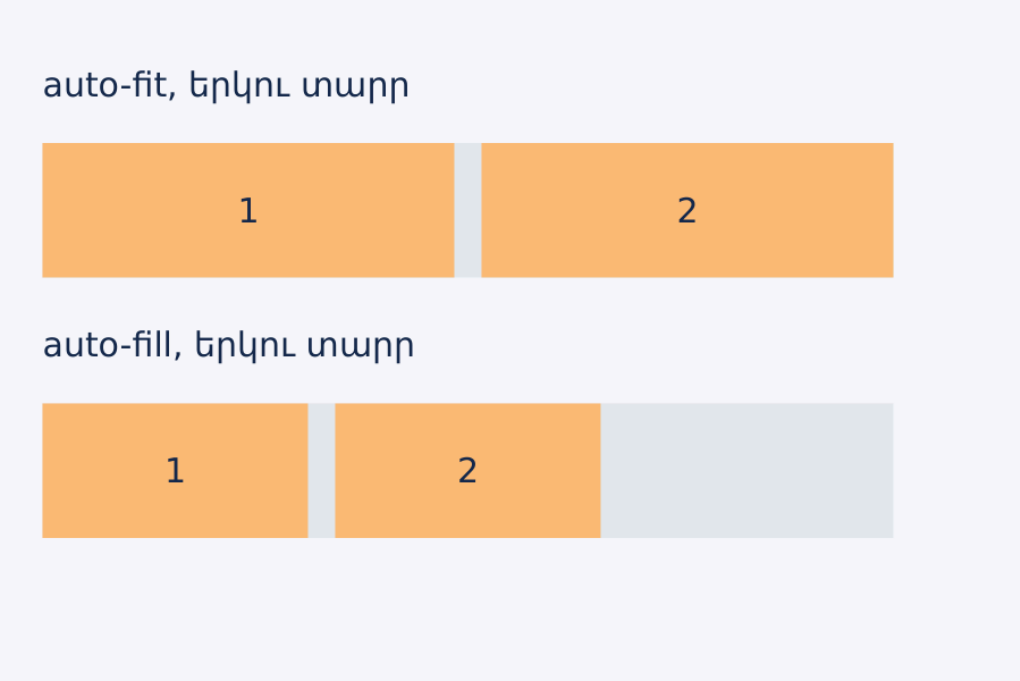
```
.grid {
  display: grid; gap: 16px;
  grid-template-columns:
    repeat(3, minmax(0, 1fr));
}
/* 3 հավասար ճկուն սյունակ */
```

auto-fit և auto-fill

auto-fit և auto-fill արժեքները հաշվարկում են, թե նվազագույն չափով քանի սյունակ կարող է տեղավորվել: Երբ բովանդակությունը բավարարում է բոլոր տեղերը, արդյունքը կարող է նույնը լինել: Տարբերությունը երևում է, երբ ցանցն ունի ավելի քիչ տարր, քան հնարավոր սյուների քանակը:

auto-fill-ը պահպանում է նաև դատարկ սյուների տեղերը: auto-fit-ը դատարկ կրկնվող սյուները փլուզում է, և զբաղված ճկուն սյուները կարող են լայնանալ: 500px լայն ծնողի և 140px նվազագույնով երկու տարրի դեպքում տարբերությունն ակնհայտ է: Այստեղ կարող էր տեղավորվել երեք սյունակ, բայց տարրերը երկուսն են: Եթե ծնողն անգամ մեկ նվազագույն սյունակից նեղ է, միայն auto-fit-ը բոլոր արտահոսքերը չի լուծում:

```
.grid {
  display: grid; gap: 16px;
  grid-template-columns:
    repeat(auto-fit,
      minmax(140px, 1fr));
}
/* փորձել նաև auto-fill */
```



Տարրի տեղադրում և span

grid-column: 1 / 3-ը տարրը տեղադրում է առաջին գծից մինչև երրորդ գիծը: Այն զբաղեցնում է երկու սյունակ, ոչ թե երեք: span 2-ը նույնպես պահանջում է երկու սյունակ, բայց սկիզբը կարող է որոշվել ավտոմատ տեղադրմամբ: grid-row-ն օգտագործում ենք նույն գաղափարով տողերի համար:

grid-template-areas-ը մեկ այլ ընթեռնելի եղանակ է էջի խոշոր հատվածները անվանելու համար: Օրինակ կարելի է գրել "header header", ապա "sidebar main": Ամեն անվանված տարածքը պետք է ուղղանկյուն լինի: Տեսանելի դիրքը փոխելը HTML հերթականությունը չի վերագրում, ուստի պետք է պահպանել տրամաբանական կառուցվածքը: Նեղ էկրանին span 2-ը հարկավոր է համադրել տվյալ պահին առկա սյուների քանակի հետ:

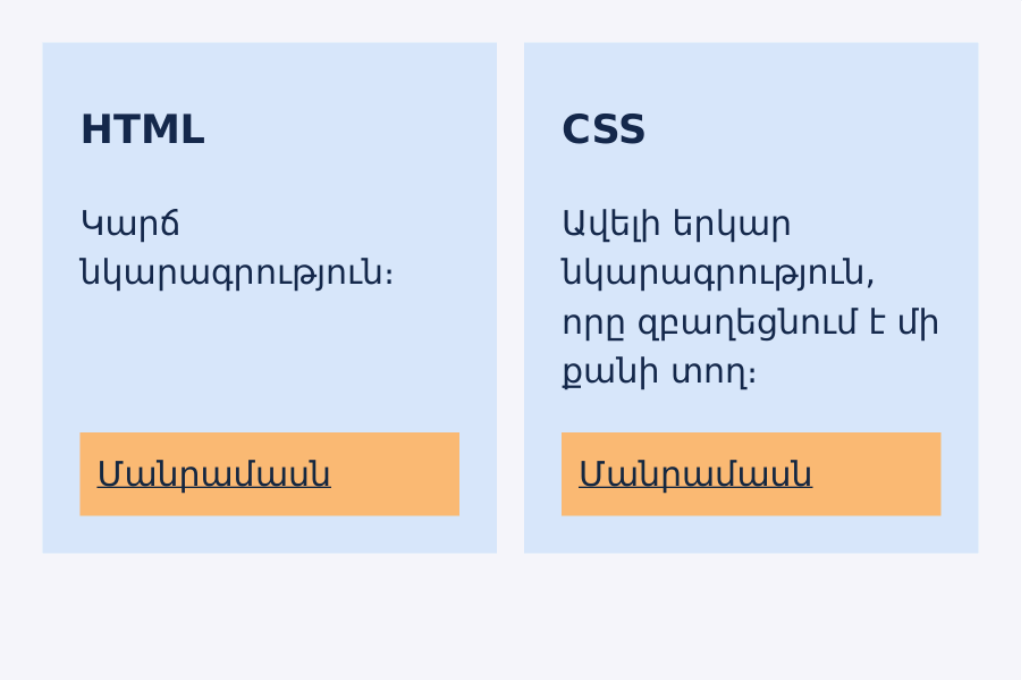
```
.featured {
  grid-column: 1 / 3;
}
/* նույն գաղափարը */
/* grid-column: span 2; */
.grid {
  grid-auto-rows: minmax(80px, auto);
}
```

Grid և Flexbox մեկ էջում

Միևնույն տարրը կարող է իր ծնողի համար լինել grid item, իսկ իր զավակների համար՝ flex container: Քարտերի ծնողին տալիս ենք Grid, որպեսզի ունենանք համաձայնեցված տողեր ու սյուներ: Յուրաքանչյուր քարտի ներսում Flexbox-ի column ուղղությամբ դասավորում ենք վերնագիրը, նկարագրությունը և հղումը:

margin-top: auto-ն հղումից վերև վերցնում է ազատ տարածքը և այն մղում ներքև: Արդյունքում նույն ցանցային տողի տարբեր նկարագրություններով քարտերի հղումները կարող են հավասարվել ներքևում: Դա ենթադրում է, որ քարտերը տողում ձգվում են հավասար բարձրության: Քարտի max-width կամ արտաքին auto margin սահմանումները կարող են խանգարել grid բջիջն ամբողջությամբ լրացնելուն:

```
.cards {
  display: grid; gap: 16px;
  grid-template-columns:
    repeat(2, minmax(0, 1fr));
}
.card {
  display: flex;
  flex-direction: column;
}
.card a { margin-top: auto; }
```



Արձագանքող վեբ դիզայն

Mobile First և viewport

Mobile First մոտեցմամբ հիմնական CSS-ը կառուցում ենք նեղ էկրանի համար, հետո min-width պայմաններով ավելացնում լայն էկրանի հնարավորությունները: Օրինակ քարտերը սկզբում մեկ սյունակ են, իսկ ավելի լայն միջավայրում դառնում են երկու կամ երեք: Սա կոդի կազմակերպման մոտեցում է, որը մեզ ստիպում է նախ մտածել կարևոր բովանդակության և պարզ դասավորության մասին:

HTML head-ի viewport meta-ն օգնում է շարժական բրաուզերին օգտագործել սարքի լայնությանը համապատասխան layout viewport: Այն ինքնուրույն էջը responsive չի դարձնում: Դրա համար դեռ անհրաժեշտ են ճկուն չափեր, հարմար պատկերներ և բովանդակությանը համապատասխան դասավորություն: Էկրանի կոտրման կետերը ընտրում ենք այնտեղ, որտեղ բովանդակությունը դադարում է հարմար տեղավորվել, ոչ միայն սարքերի անուններով:

```
<meta name="viewport"
  content="width=device-width,
  initial-scale=1">

.cards {
  display: grid; gap: 16px;
  grid-template-columns: 1fr;
}
```

Media Queries և կոտրման կետեր

Media Query-ն CSS կանոնները կապում է միջավայրի պայմանին: Այս օրինակում մինչև 640px լայնության քարտերը մեկ սյունակ են: 640px-ից սկսած գործում է երկու սյունակ սահմանող կանոնը: 1000px-ից սկսած երկու պայմանն էլ ճշմարիտ են, բայց նույն ընտրիչի ավելի ուշ գրված կանոնը տալիս է երեք սյունակ:

Հասկացությունը ամրապնդելու համար ստուգում ենք 639, 640, 999 և 1000px լայնությունները: min-width-ը ներառում է հենց շեմը: max-width-ը գործում է մինչև նշված շեմը ներառյալ: Նույն Media Query-ի ներսում գործում են սովորական կասկադի և առաջնահերթության կանոնները: Եթե ընտրիչների կշիռները տարբեր են, միայն կողի ուշ տեղադրումը միշտ բավարար չէ: Այս թվերը օրինակային կոտրման կետեր են և պարտադիր ստանդարտ շեմեր չեն:

```
.cards { grid-template-columns: 1fr; }
@media (min-width: 640px) {
  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}
@media (min-width: 1000px) {
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Ճկուն չափեր և պատկերներ

Ֆիքսված մեծ լայնությունը կարող է էջին հորիզոնական ոլորում տալ: Այստեղ container-ի լայնությունը ընտրում է 1100px-ը կամ հասանելի լայնությունից 32px պակասը՝ որն ավելի փոքր է: margin-inline: auto-ն այն կենտրոնացնում է: Փոքր էկրանին երկու կողմերում մնում է ընդհանուր 32px ազատ տարածք:

Պատկերին max-width: 100% և height: auto տալով պահպանում ենք համամասնությունները և թույլ չենք տալիս, որ այն գերազանցի ծնողին: Եթե քարտերում բոլոր պատկերները պետք է ունենան նույն բարձրությունը, կարող ենք կիրառել width: 100%, height: 160px և object-fit: cover: Այդ դեպքում պատկերի մի մասը կարող է կտրվել: Պատկերների HTML width և height հատկանիշները օգնում են նախապես տեղ պահել: alt տեքստը նկարագրում է բովանդակային պատկերի իմաստը:

```
.container {
  width: min(1100px, 100% - 32px);
  margin-inline: auto;
}
img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

Responsive էջի ստուգում

Responsive ստուգումը միայն հեռախոս, պլանշետ և համակարգիչ երեք կոճակ սեղմելը չէ: Բրաուզերի լայնությունը փոխում ենք շարունակաբար: Փորձարկում ենք երկար վերնագիր, երկար URL, շատ մենյուի հղումներ և մեծացված տեքստ: Եթե բովանդակությունն արտահոսում է, պարզում ենք կոնկրետ տարրը՝ ֆիքսված լայնություն, չնեղացող flex item, մեծ պատկեր կամ չկոտրվող բառ:

360, 768 և 1280px լայնությունները ներկայացնում են տարբեր չափերի միջավայրեր, սակայն դրանց միջև գտնվող լայնություններում ևս դասավորությունը պետք է մնա ընթեռնելի: Ամբողջ body-ին overflow-x: hidden տալը կարող է պարզապես թաքցնել խնդիրը և բովանդակությունը: Լայն աղյուսակի համար ավելի օգտակար է առանձին ոլորվող պատյան: Փորձարկում ենք նաև Tab-ով հղումները, focus-ի տեսանելիությունը և sticky տարրերի՝ բովանդակությունը չծածկելը:

```
/* Երկար բառ կամ URL */
.card { min-width: 0; }
.card p { overflow-wrap: anywhere; }

/* Լայն աղյուսակ */
.table-wrap { overflow-x: auto; }
```

CSS Էֆեկտներ և Sass SCSS

CSS transition

Transition-ը սահմանում է, թե ինչպես է հատկությունը մի արժեքից անցնում մյուսին: Միայն transition գրելը բավարար չէ. պետք է նաև արժեքի փոփոխություն, օրինակ hover կամ focus-visible վիճակում: Կարճ գրառումը սովորաբար ներառում է հատկությունը, տևողությունը, ժամանակային ֆունկցիան և ցանկության դեպքում ուշացումը:

Օրինակում փոխվում են ֆոնն ու transform-ը: Սկզբնական գույնը պետք է արդեն սահմանված լինի: Transition-ը գրում ենք հիմնական ընտրիչում, որպեսզի թե՛ hover մտնելիս, թե՛ դուրս գալիս փոփոխությունը լինի մեղմ: ease-ը փոխում է ընթացքի արագությունը, linear-ը պահում է հավասար ժամանակային առաջընթաց: Խուսափում ենք transition: all-ը որպես մշտական սովորություն կիրառելուց. նշում ենք այն հատկությունները, որոնք իսկապես պետք է շարժվեն: Ստեղնաշարով օգտագործողը նույնպես պետք է տեսնի հստակ focus:

```
.button {
  transition: background-color .25s ease,
              transform .25s ease;
}
.button:hover,
.button:focus-visible {
  background-color: #f97316;
  transform: translateY(-4px);
}
```

Transform և տեսանելի դիրք

Transform-ը փոխում է տարրի տեսանելի պատկերը: translate-ը տեղափոխում է, rotate-ը պտտում, scale-ը փոխում է մասշտաբը, skew-ը շեղում է: Տարրի սկզբնական դասավորության տարածքը պահպանվում է, հետևաբար մեծացած կամ պտտված տարրը կարող է ծածկել հարևանին: Տեսանելի սահմանները և overflow-ը կարող են փոխվել:

Նույն չափերի տարրերի վրա տարբեր transform ֆունկցիաները տալիս են տարբեր տեսողական արդյունքներ: Տարրի լայնության փոփոխությունն ու scale-ը տարբեր ազդեցություն ունեն

դասավորության վրա: Width-ի փոփոխությունը սովորաբար ազդում է դասավորության վրա, իսկ transform-ը հիմնականում փոխում է արդեն դասավորված տարրի պատկերը: Transform ունեցող տարրը կարող է նաև ստեղծել stacking context և փոխել absolute կամ fixed զավակների հենակետը: Սա կապում ենք նախորդ դիրքավորման թեմայի հետ:

```
.a { transform: translateX(40px); }
.b { transform: rotate(12deg); }
.c { transform: scale(1.15); }
.d { transform: skewX(-12deg); }
```

Transform հերթականություն և հենակետ

Transform ֆունկցիաները միավորելիս կարևոր է դրանց հերթականությունը: Մատրիցների բազմապատկման տեսանկյունից աջ կողմի գործողությունն ազդում է առաջինը: translateX(100px) rotate(45deg) և rotate(45deg) translateX(100px) գրառումները նույն պատկերը չեն տալիս, որովհետև պտտումը փոխում է նաև ուղղությունների հարաբերությունը:

transform-origin-ը սահմանում է փոխակերպման հենակետը: Լռելյայն կենտրոնի փոխարեն left top վերցնելիս պտտումը տեղի է ունենում վերին ձախ անկյան շուրջ: Առանձին փոխակերպումների համադրությունը ձևավորում է մեկ ընդհանուր փոխակերպում, որի արդյունքը կախված է ինչպես արժեքներից, այնպես էլ ֆունկցիաների հերթականությունից:

```
.a {
  transform: translateX(100px) rotate(45deg);
}
.b {
  transform: rotate(45deg) translateX(100px);
}
.hinge { transform-origin: left top; }
```

Keyframes անիմացիա

Անիմացիայի համար նախ անվանում ենք փուլերը՝ @keyframes enter: from և to արժեքները համապատասխանում են 0% և 100% կետերին: Հետո այդ անունը կապում ենք տարրին animation հատկությամբ: Օրինակում քարտը միաժամանակ դառնում է տեսանելի և բարձրանում 20px-ով:

Transition-ը սովորաբար արձագանքում է արժեքի փոփոխությանը: Keyframes անիմացիան կարող է սկսել, երբ ոճը կիրառվում է՝ առանց hover-ի: Կարող ենք ունենալ նաև միջանկյալ տոկոսային փուլեր: both-ը պահում է առաջին փուլի ազդեցությունը ուշացման ընթացքում և վերջինինը ավարտից հետո: Բրաուզերը միջանկյալ արժեքները հաշվարկում է ժամանակի ընթացքում՝ ստեղծելով անընդհատ շարժման տպավորություն: Կարճ և հանգիստ մուտքի անիմացիան կարող է ընդգծել տարրի հայտնվելը՝ առանց ընթերցմանը խանգարելու:

```
@keyframes enter {
  from { opacity: 0;
    transform: translateY(20px); }
  to { opacity: 1;
    transform: translateY(0); }
}
.card {
  animation: enter .6s ease both;
}
```

Animation պարամետրեր

animation-duration-ը մեկ շրջանի տևողությունն է, delay-ը՝ մեկնարկի սպասումը: iteration-count-ը որոշում է կրկնությունների թիվը կամ infinite անսահման կրկնությունը:

direction: alternate-ը յուրաքանչյուր հերթական շրջանում փոխում է ուղղությունը: Այս երեք կրկնության օրինակում առաջին շրջանը առաջ է, երկրորդը՝ հետ, երրորդը՝ կրկին առաջ:

fill-mode-ը որոշում է՝ արդյոք փուլերի ոճերն ազդում են ակտիվ միջակայքից դուրս: forwards-ը պահում է ավարտի փուլը, backwards-ը՝ ուշացման ընթացքում մեկնարկայինը, both-ը՝ երկուսն էլ: play-state: paused-ը ժամանակավորապես կանգնեցնում է ընթացքը: Կողին անհրաժեշտ է նաև pulse անունով @keyframes սահմանում, օրինակ from { transform: scale(1) } to { transform: scale(1.15) }: Անընդհատ շարժումը ընտրում ենք միայն այն ժամանակ, երբ այն օգնում է օգտատիրոջը:

```
.dot {
  animation-name: pulse;
  animation-duration: 1.4s;
  animation-delay: .2s;
  animation-iteration-count: 3;
  animation-direction: alternate;
  animation-fill-mode: both;
  animation-timing-function: ease-in-out;
}
.dot:hover { animation-play-state: paused; }
```

Շարժման նվազեցման նախընտրություն

Օգտատերը համակարգում կարող է նշել, որ նախընտրում է քիչ շարժում: prefers-reduced-motion պայմանով կարող ենք փոխել էջի անիմացիաներն ու անցումները: Այս օրինակում անջատում ենք քարտի մուտքն ու կոճակի տեղաշարժը, բայց թողնում ենք գույնի և focus-ի տեսանելի տարբերությունը:

Կարևոր է, որ սովորական վիճակում քարտը արդեն լինի տեսանելի: Եթե opacity: 0-ը գրված է հիմնական .card կանոնում և միայն անիմացիան է այն դարձնում 1, անիմացիան անջատելուց հետո բովանդակությունը կարող է անհետ մնալ: Այդ պատճառով օրինակում թափանցիկությունը սահմանում ենք միայն keyframes-ի ներսում: Reduced motion նախընտրության դեպքում բովանդակությունն ու գործողությունները պետք է պահպանեն իրենց հասանելիությունը՝ առանց անիմացիայից կախված լինելու:

```
@media (prefers-reduced-motion: reduce) {
  .card { animation: none; }
  .button {
    transition: none;
  }
  .button:hover,
  .button:focus-visible {
    transform: none;
  }
}
```

CSS ֆիլտրներ

Ֆիլտրներով կարող ենք փոխել պատկերի գունային կամ տեսողական հատկությունները: grayscale(1)-ը ամբողջովին մոխրագույն է դարձնում պատկերը, blur-ը լղոզում է, brightness-ը փոխում է պայծառությունը, contrast-ը՝ հակադրությունը: Մի քանի ֆիլտր կարելի է գրել մեկ filter արժեքի մեջ: Նրանց հերթականությունը կարող է ազդել արդյունքի վրա:

Օրինակում պատկերը սկզբում մոխրագույն է, hover-ի ժամանակ վերադառնում է գույներին: Transition-ը դարձնում է այդ փոփոխությունը մեղմ: Ֆիլտրները կարող են ազդել նաև տեքստ պարունակող ամբողջ տարրի վրա, ուստի ավելի լավ է դրանք կիրառել կոնկրետ պատկերի կամ նպատակային բլոկի վրա: drop-shadow()-ը ստվերում է արտապատկերված պատկերի ալֆա

ուրվագիծը, մինչդեռ box-shadow-ը վերաբերում է բլոկի տուփին: Լողոցումը գաղտնի տվյալը թաքցնելու անվտանգ միջոց չէ:

```
.photo {
  filter: grayscale(1);
  transition: filter .3s ease;
}
.photo:hover { filter: grayscale(0); }

/* blur(3px), brightness(1.2) */
/* contrast(1.3), sepia(1) */
```

Բազմասյուն տեքստ

Բազմասյուն կառուցվածքը նման է թերթի կամ ամսագրի տեքստին: Այն մեկ շարունակական բովանդակությունը բաժանում է սյուների: Grid-ում մենք սովորաբար տեղադրում ենք անկախ տարրեր՝ ըստ բջիջների: Այստեղ ընթերցումը շարունակվում է առաջին սյունակի ներքևից հաջորդ սյունակի վերևը:

column-count-ը տալիս է սյուների ցանկալի քանակը, column-width-ը՝ ցանկալի լայնությունը: Կարելի է գրել columns: 240px 3՝ թույլ տալով միջավայրին սահմանափակել քանակը հասանելի տարածքով: column-gap-ը բացատն է, column-rule-ը՝ բաժանարար գիծը: column-span: all-ը վերնագիրը կարող է տարածել բոլոր սյուների վրա: break-inside: avoid-ը խնդրում է խուսափել նկարի կամ այլ բլոկի ներսում ընդհատումից, բայց բոլոր դեպքերի համար անխախտ երաշխիք չէ: Նեղ էկրանին հաճախ վերադառնում ենք մեկ սյան:

```
.article {
  column-count: 2;
  column-gap: 28px;
  column-rule: 1px solid #94a3b8;
}
.article h2 { column-span: all; }
.article figure { break-inside: avoid; }
```

Վեբ տեխնոլոգիաներ

HTML-ը սահմանում է էջի կառուցվածքը։ CSS-ը ձևավորում է դրա տեսքն ու դասավորությունը։ Ուսանողը ստեղծում է իմաստային կառուցվածք։

Հետո ավելացնում ենք գույներ, բացատներ և չափեր։ Էջը ստուգում ենք տարբեր լայնություններով և պահպանում ընթեռնելիությունը։

Sass և SCSS շարահյուսություն

Sass-ը ոճաթերթերի նախամշակման լեզու և գործիքակազմ է: Այն ունի երկու շարահյուսություն՝ SCSS և նահանջներով գրվող Sass: SCSS ֆայլը օգտագործում է ձևավոր փակագծեր և կետ-ստորակետեր, իսկ .sass տարբերակը՝ նահանջներ: Երկուսն էլ մշակումից հետո տալիս են սովորական CSS:

Բրաուզերին միացնում ենք ստացված style.css ֆայլը: Միայն .scss ֆայլը link անելն այն Sass մշակիչ չի դարձնում: SCSS-ը ուսանողների համար հարմար մեկնարկ է, որովհետև սովորած CSS կանոնները կարելի է տեղափոխել այդ ֆայլի մեջ և աստիճանաբար ավելացնել Sass հնարավորությունները: Ժամանակակից CSS-ն արդեն ունի որոշ նման հնարավորություններ, օրինակ փոփոխականներ և nesting, բայց Sass-ի մշակման փուլը և սեփական կառուցվածքները շարունակում են տարբեր լինել:

```
/* style.scss */
$accent: #f97316;
.button {
  background: $accent;
}

/* style.sass */
$accent: #f97316
.button
  background: $accent
```

Մշակման արդյունքում ստացվող CSS-ը հետևյալն է:

```
.button {
  background: #f97316;
}
```

Sass և CSS փոփոխականներ

Sass-ի $space փոփոխականը մշակման ժամանակ փոխարինվում է արժեքով: Վերջնական CSS-ում padding: 16px է մնում: CSS custom property-ն՝ --accent-ը, մնում է վերջնական CSS-ում և աշխատում է բրաուզերի կասկադով: Այն կարող ենք վերասահմանել որոշակի տարրի կամ թեմայի համար:

Օրինակում բացատի չափը պահում ենք Sass փոփոխականի մեջ, իսկ թեմայի գույնը՝ CSS փոփոխականի: Եթե քարտը գտնվում է dark դաս ունեցող տարրի ներսում, կարող է ժառանգել նոր --accent արժեքը: $space և --accent անունները նման խնդիրներ են լուծում տարբեր փուլերում: Sass փոփոխականը ինքնուրույն չի փոխվի, երբ օգտատերը բացի մութ ռեժիմը: Կողի յուրաքանչյուր փոփոխությունից հետո հարկավոր է նորից մշակել SCSS-ը կամ գործարկել watch ռեժիմը:

```
$space: 16px;
:root { --accent: #f97316; }
.card {
  padding: $space;
  color: var(--accent);
}
.dark { --accent: #fdba74; }
```

Մշակման արդյունքում ստացվող CSS-ը հետևյալն է:

```
.card {
  padding: 16px;
  color: var(--accent);
}
```

Nesting և ampersand

SCSS nesting-ը թույլ է տալիս ընտրիչները գրառել միմյանց ներսում: Օրինակում .nav-ի ներսի a-ն մշակումից հետո դառնում է .nav a: &:hover-ի ampersand-ը հղվում է ընթացիկ ամբողջ ընտրիչին, հետևաբար ստացվում է .nav a:hover: Առանց & նշանի գրված :hover-ը կարող էր նշանակել այլ ժառանգ ընտրիչ:

Ներդրվածությունը պետք է օգնի հասկանալ կառուցվածքը: Եթե մի քանի մակարդակ շարունակ կրկնում ենք HTML-ի բոլոր նախնիներին, ստանում ենք երկար և բարձր կշռով ընտրիչներ: Ավելի լավ է պահել քիչ մակարդակներ և օգտագործել պարզ դասեր: Ժամանակակից CSS nesting-ն ու SCSS nesting-ը որոշ հնարավորություններով նման են, բայց դրանք պետք չէ առանց ստուգման նույն լեզուն համարել: Այս օրինակում խոսքը հստակ SCSS մշակման մասին է:

```
.nav {
  a {
    color: navy;
    &:hover { color: coral; }
  }
}
```

Մշակման արդյունքում ստացվող CSS-ը հետևյալն է:

```
.nav a {
  color: navy;
}
.nav a:hover {
  color: coral;
}
```

Mixin և կրկնվող կանոններ

Mixin-ը անվանված կանոնների խումբ է, որը տեղադրում ենք @include-ով: Այն կարող է ընդունել պարամետրեր և ունենալ լռելյայն արժեքներ: Այս օրինակում row mixin-ը կենտրոնացնում է տարրերը լայնական ուղղությամբ և թույլ է տալիս ընտրել gap-ը: Nav-ը ստանում է 24px, իսկ meta-ն՝ լռելյայն 16px:

Մշակիչը կանոնները տեղադրում է յուրաքանչյուր օգտագործման վայրում: Սա հարմար է կրկնությունը սկզբնական ֆայլում նվազեցնելու համար, բայց վերջնական CSS-ում կանոնները կարող են կրկնվել: Ուստի ամեն մի փոքր հայտարարության համար mixin ստեղծելը անհրաժեշտ չէ: Այն օգտակար է, երբ ունենք մի քանի կապված հայտարարություններ կամ պարամետրով փոփոխվող կրկնվող լուծում: Տարբեր պարամետրերով նույն mixin-ի կիրառումը ստեղծում է նույն կառուցվածքով, բայց տարբեր արժեքներով CSS կանոններ:

```
@mixin row($gap: 16px) {
  display: flex;
  align-items: center;
  gap: $gap;
}
.nav { @include row(24px); }
.meta { @include row; }
```

Մշակման արդյունքում ստացվող CSS-ը հետևյալն է:

```
.nav {
  display: flex;
  align-items: center;
  gap: 24px;
}
.meta {
  display: flex;
  align-items: center;
  gap: 16px;
}
```

Մողուլներ և SCSS մշակում

Մեծացող նախագծում գույները, բացատներն ու կրկնվող կանոնները կարելի է բաժանել ֆայլերի: _tokens.scss-ի սկզբի ընդգծումը նշում է partial ֆայլ: style.scss-ը @use "tokens" գրառմամբ բեռնում է այն, ապա tokens.$accent ձևով դիմում փոփոխականին: Namespace-ը օգնում է տեսնել՝ որտեղից է եկել անունը և կանխել պատահական բախումները:

Ժամանակակից Sass նախագծում մողուլների համար օգտագործում ենք @use, ոչ թե նոր կողը կառուցում հնացած Sass @import-ի վրա: Եթե Dart Sass-ը տեղադրված է, sass style.scss style.css հրամանը մեկ անգամ մշակում է ֆայլը: --watch տարբերակը հետևում է փոփոխություններին և նորից մշակում: Եթե հրամանը չի ճանաչվում, նախ պետք է տեղադրել Dart Sass-ը կամ նախագծի գործիքով գործարկել այն: HTML-ում մնում է link դեպի style.css:

```
/* _tokens.scss */
$accent: #f97316;

/* style.scss */
@use "tokens";
.button { color: tokens.$accent; }

/* հրամանային տող */
sass --watch style.scss style.css
```

Միավորված էջի կառուցվածք

Ուսումնական կենտրոնի էջը կարող է միավորել այս դասավորության միջոցները մեկ կառուցվածքում: Կպչուն նավիգացիայի ներսում Flexbox-ը բաշխում է կայքի անունն ու հղումները: Քարտերի ընդհանուր բաժինը Grid է: Յուրաքանչյուր քարտի ներսում Flexbox-ը պահում է հղումը ներքևում, իսկ relative և absolute զույգը տեղադրում է Նոր նշանը:

Սկզբնական մեկ սյունակը min-width Media Query-ներով դառնում է երկու և երեք: Հղման hover կամ focus վիճակում ավելացնում ենք մեղմ գունային անցում: Միայն զարդարող մուտքի շարժումը անջատում ենք reduced motion նախընտրության դեպքում: Այս մոտեցմամբ յուրաքանչյուր ծնող տարր կառավարում է իր անմիջական զավակների դասավորությունը, իսկ էջի ընդհանուր կառուցվածքը կազմվում է այդ մակարդակների համագործակցությամբ:

```
.nav { display: flex; flex-wrap: wrap; }
.cards { display: grid; gap: 16px; }
.card {
  position: relative;
  display: flex; flex-direction: column;
}
.badge { position: absolute; }
.card a { margin-top: auto; }
```

Ամբողջական էջի օրինակ

Հետևյալ օրինակում HTML կառուցվածքը և CSS կանոնները գտնվում են նույն փաստաթղթում: Նավիգացիան օգտագործում է Flexbox, քարտերի բաժինը՝ Grid, իսկ յուրաքանչյուր քարտի ներքին կառուցվածքը կրկին Flexbox է: Կոտրման կետերը փոխում են սյուների քանակը, իսկ շարժման նվազեցման նախընտրությունը անջատում է ոչ պարտադիր շարժումները:

```
<!doctype html>
<html lang="hy">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Ուսումնական կենտրոն</title>
<style>
* { box-sizing: border-box; }
:root { --accent: #f97316; }
body { margin: 0; font: 16px/1.5 'DejaVu Sans', sans-serif;
  color: #172b4d; background: #f1f5f9; }
a { color: #1d4ed8; }
a:focus-visible { outline: 3px solid var(--accent); outline-offset: 4px; }
.container { width: min(1100px, 100% - 32px); margin-inline: auto; }
.nav { position: sticky; top: 0; z-index: 10; background: white;
  display: flex; flex-wrap: wrap; align-items: center;
  justify-content: space-between; gap: 16px; padding: 16px; }
.links { display: flex; flex-wrap: wrap; gap: 16px;

  list-style: none; margin: 0; padding: 0; }
.cards { display: grid; grid-template-columns: 1fr; gap: 16px; }
.card { position: relative; display: flex; flex-direction: column;
  min-width: 0; padding: 24px; background: white;
  border: 1px solid #cbd5e1; border-radius: 12px;
  animation: enter .6s ease both; }
.card p { overflow-wrap: anywhere; }
.card a { align-self: flex-start; margin-top: auto; padding: 10px 16px;
  background: #dbeafe; transition: background-color .25s, transform .25s; }
.card a:hover, .card a:focus-visible {
  background: #fed7aa; transform: translateY(-3px); }
.badge { position: absolute; top: 12px; right: 12px;
  background: var(--accent); color: #172b4d; padding: 2px 10px; }
#about { margin-block: 32px; }
footer { padding-block: 24px; }
@media (min-width: 640px) {
  .cards { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
```

```
@media (min-width: 1000px) {
  .cards { grid-template-columns: repeat(3, minmax(0, 1fr)); }
}
@keyframes enter {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}
@media (prefers-reduced-motion: reduce) {
  .card { animation: none; }
  .card a { transition: none; }
  .card a:hover, .card a:focus-visible { transform: none; }
}
</style>
</head>
<body>
<nav class="nav" aria-label="Հիմնական նավիգացիա">
<strong>WEB դպրոց</strong>
<ul class="links">

<li><a href="#courses">Դասեր</a></li>
<li><a href="#about">Մեր մասին</a></li>
<li><a href="#contact">Կապ</a></li>
</ul>
</nav>
<main class="container">
<h1>Վեբ դասընթացներ</h1>
<section id="courses" aria-label="Դասընթացներ" class="cards">
<article class="card"><span class="badge">Նոր</span>
<h2>HTML</h2><p>Էջի իմաստային կառուցվածք և ձևեր:</p>
<a href="#contact">Գրանցվել HTML-ին</a></article>
<article class="card"><h2>CSS</h2>
<p>Գույներ, բացատներ, դիրքավորում, Flexbox և Grid:</p>
<a href="#contact">Գրանցվել CSS-ին</a></article>
<article class="card"><h2>JavaScript</h2><p>Ինտերակտիվ էջի հիմունքներ:</p>
<a href="#contact">Գրանցվել JavaScript-ին</a></article>
<article class="card"><h2>Responsive</h2><p>Էջի հարմարեցում տարբեր էկրաններին:</p>
<a href="#contact">Գրանցվել Responsive-ին</a></article>

<article class="card"><h2>SCSS</h2><p>Փոփոխականներ և կրկնվող ոճերի կառավարում:</p>
<a href="#contact">Գրանցվել SCSS-ին</a></article>
<article class="card"><h2>Git</h2><p>Նախագծի փոփոխությունների պահպանում:</p>
<a href="#contact">Գրանցվել Git-ին</a></article>
</section>
<section id="about"><h2>Մեր մասին</h2>
<p>Սովորում ենք օրինակներով և կառուցում սեփական կայքը:</p></section>
</main>
<footer id="contact" class="container">Գրանցման հարցերով՝
<a href="mailto:school@example.com">school@example.com</a></footer>
</body>
</html>
```

Աղբյուրներ

https://www.w3.org/TR/css-position-3/

https://www.w3.org/TR/CSS22/visuren.html#floats

https://www.w3.org/TR/css-flexbox-1/

https://www.w3.org/TR/css-grid-2/

https://www.w3.org/TR/mediaqueries-5/

https://www.w3.org/TR/css-transitions-1/

https://www.w3.org/TR/css-transforms-1/

https://www.w3.org/TR/css-animations-1/

https://www.w3.org/TR/filter-effects-1/

https://www.w3.org/TR/css-multicol-1/

https://sass-lang.com/guide/

https://sass-lang.com/documentation/at-rules/use/

https://sass-lang.com/documentation/at-rules/mixin/

https://sass-lang.com/documentation/cli/dart-sass/